UNIVERSIDAD CATÓLICA DE EL SALVADOR
FACULTAD DE INGENIERÍA Y ARQUITECTURA

"La Ciencia sin Moral es Vana"

FASES DE PLANIFICACIÓN Y DISEÑO

DESARROLLO DE APLICACIONES MÓVILES II · SECCIÓN A

PRESENTADO POR
BRANDON STANLEY MONZÓN AGREDA
LUIS MARIO BENITEZ DOMINGUEZ
JUAN JOSÉ MAGAÑA MOREIRA
JOSÉ EDUARDO RAMOS LÓPEZ

DOCENTE: ING. JOSÉ RAÚL RODRÍGUEZ FIGUEROA

SANTA ANA, 03 DE MARZO DE 2026

INTRODUCCIÓN

El consumo de contenido audiovisual ha experimentado un crecimiento significativo en los últimos años, impulsado por la expansión de dispositivos móviles y plataformas digitales. Sin embargo, muchas aplicaciones actuales como plataformas de streaming combinan múltiples funciones (reproducción, pagos, perfiles, descargas), lo que puede generar complejidad innecesaria para usuarios que únicamente desean consultar información sobre películas.

Ante esta necesidad, se propone el desarrollo de una aplicación móvil nativa para iOS denominada CineInfo, que permita explorar, buscar y guardar información de películas de forma rápida, intuitiva y ligera.

El presente documento describe detalladamente las fases de Planificación y Diseño del proyecto, aplicando la metodología Mobile Sprint Model (MMS), la cual permite estructurar el desarrollo en ciclos iterativos, asegurando control, calidad y mejora continua.

OBJETIVOS

Objetivo General

Planificar y diseñar una aplicación móvil nativa para iOS que permita explorar información de películas mediante una arquitectura Cliente–Servidor, aplicando la metodología Mobile Sprint.

Objetivos Específicos

- Analizar el entorno tecnológico y académico del proyecto.
- Definir el alcance funcional y no funcional.
- Identificar los stakeholders involucrados.
- Evaluar la factibilidad técnica, económica y operativa.
- Planificar la ejecución por sprints.
- Diseñar la arquitectura general del sistema.
- Definir el modelo de datos y estructura de base de datos.
- Establecer mecanismos de gestión de calidad y control de cambios.

FASE DE PLANIFICACIÓN

La fase de planificación es fundamental dentro del modelo MMS, ya que establece las bases estructurales del proyecto antes de iniciar el desarrollo técnico.

Objetivos de la Fase de Planificación

- Definir claramente el problema a resolver.
- Establecer los requerimientos del sistema.
- Delimitar el alcance del proyecto.
- Analizar riesgos potenciales.
- Estimar tiempo y recursos.
- Diseñar la estrategia de desarrollo por sprints.

¿Quiénes Intervienen?

Los siguientes actores participan activamente en la fase de planificación:

Stakeholder	Rol	Nivel de Interés	Nivel de Influencia
Docente	Supervisor académico	Alto	Alto
Equipo desarrollador	Desarrollo y diseño	Alto	Alto
Usuarios finales	Uso de la aplicación	Medio	Bajo
Administrador Backend	Gestión de base de datos	Medio	Medio

Estrategia de Gestión

- Reuniones semanales de seguimiento.
- Entregables por sprint.
- Retroalimentación continua.
- Presentación de avances parciales.

Actividades en la Fase de Planificación

- Identificación y análisis del problema.
- Levantamiento de requerimientos funcionales y no funcionales.
- Identificación de stakeholders y gestión de interesados.
- Definición del entorno tecnológico.
- Elaboración del modelo de negocio.
- Análisis de factibilidad técnica, económica y operativa.
- Planificación de sprints y estructura de desglose de trabajo (EDT).
- Definición de la gestión de alcance, tiempo, costos, cambios y calidad.

Entorno de la Empresa

Entorno Tecnológico

Componente	Detalle
Frontend – Lenguaje	Swift
Frontend – Framework	SwiftUI
Frontend – IDE	Xcode
Frontend – Arquitectura	MVC
Backend – Lenguaje	Python
Backend – Framework	Flask
Backend – Base de Datos	MySQL
Backend – Arquitectura	RESTful API

Entorno Académico

- Proyecto desarrollado con fines educativos.
- Supervisión directa del docente.
- Evaluación por entregables parciales.

Modelo de Negocio

Tipo de Aplicación	Aplicación gratuita de consulta informativa.
Propuesta de Valor	Brindar una experiencia ligera, rápida y clara para consultar información cinematográfica sin distracciones.
Público Objetivo	Usuarios que desean consultar datos de películas sin necesidad de suscripción o reproducción.

Análisis de Factibilidad

1. Factibilidad Técnica

El proyecto es viable por las siguientes razones:

- Swift es el lenguaje oficial para iOS.
- SwiftUI permite el desarrollo de interfaces modernas y adaptativas.
- Flask facilita la creación de APIs REST de forma rápida.
- MySQL garantiza integridad y consistencia de los datos.
- No se requiere hardware especializado más allá de un equipo Mac con Xcode.

▸ **Resultado: Alta viabilidad técnica.**

2. Factibilidad Económica

Al tratarse de un proyecto académico:

- No requiere inversión en licencias comerciales.
- Uso exclusivo de herramientas gratuitas.
- El hosting puede realizarse en servidor local o gratuito.

Concepto	Costo Estimado
Desarrollo	$0
Hosting opcional	$50 – $60 anual
Base de datos	$0

▸ **Resultado: Viable económicamente.**

3. Factibilidad Operativa

- Interfaz intuitiva de fácil adopción.
- No requiere capacitación especializada.
- Uso sencillo mediante navegación básica.

▸ **Resultado: Operativamente viable.**

Resultados Esperados

- Aplicación funcional en iOS.
- Sistema de autenticación seguro.
- Consulta dinámica de películas vía API externa.
- Gestión de favoritos persistente en base de datos.
- Backend estable con API REST operativa.
- Documentación completa del proyecto.

Gestión de Stakeholders

Matriz General de Stakeholders

Interesado	Rol	Responsabilidad
Docente	Supervisor de Proyecto	Definición de requerimientos, auditoría de código y validación de fases.
Estudiantes	Equipo Full-stack	Desarrollo integral de interfaces, servicios de red y persistencia.
Usuarios Finales	Consumidores	Validación de la fluidez de la interfaz y la facilidad de búsqueda.

Matriz Específica por Fases

Rol	Actividades	Nivel	Acceso a Información
Desarrolladores	Implementación técnica y gestión del repositorio.	5	Total (Código, API y BD)
Cliente	Supervisión de avances y retroalimentación técnica.	4	Código fuente y bitácoras

Matriz de Actividades

Tarea	Descripción
Configuración de Repositorio	Creación de entorno colaborativo en GitHub/GitLab para el equipo.
Integración de API	Conexión con servicio externo (TMDB o similar) para catálogo.
Lógica de App	Desarrollo de vistas responsivas en SwiftUI.

Gestión del Alcance

Requerimientos Funcionales

Requerimiento	Descripción	Observación
Registro	Creación de cuenta.	Almacenamiento en base de datos local del servidor.
Login	Validación de acceso al sistema.	Habilita la persistencia de sesión en el dispositivo.
Exploración	Visualización de datos desde API.	Los datos se procesan a través del puente en Flask.
Favoritos	Almacenamiento de películas seleccionadas.	Guarda ID externo, título, imagen y descripción en MySQL.

Especificación de Requerimientos Funcionales

Responsable	Equipo de Desarrollo
Actor	Usuario Final
Pre-condición	El usuario debe iniciar sesión exitosamente en la aplicación.
Post-condición	Las credenciales se conservan localmente para accesos futuros.
Descripción	Una vez validado el ingreso, la aplicación almacena los datos de sesión para evitar cierres de sesión involuntarios al reiniciar la app.

Requerimientos No Funcionales

Atributo	Descripción	Observación
Desempeño	Tiempo de respuesta ágil en consultas asíncronas.	Gestión de hilos para no bloquear la interfaz.
Experiencia de Usuario	Navegación intuitiva con diseño responsivo.	Adaptación del contenido a orientaciones Portrait y Landscape.
Manejo de Errores	Notificación mediante ventanas emergentes (Pop-ups).	Mensajes claros en caso de fallos de red o API.

Historias de Usuario

Identificador	HU-01
Actor	Usuario Final
Prioridad	Alta
Sprint	Bloque 3
Responsable	Equipo de Desarrollo
Requerimiento Funcional	Consulta de Catálogo
Observaciones	Grid responsivo que se ajusta dinámicamente al tamaño de pantalla.
Bosquejos / Prototipos	Vista de Rejilla Adaptativa

Estructura de Desglose de Trabajo (EDT)

- Ejecución: Creación de repositorio, integración de API externa y desarrollo SwiftUI.
- Pruebas: Validación de integración local y depuración de errores.
- Lanzamiento: Compilación final en hardware institucional y entrega de informes.

Gestión del Tiempo

Periodo	Semanas	Actividad Principal
Bloque 1	1 – 2	Fase 3: Creación de repositorio, setup de servidor y sistema de registro.
Bloque 2	3 – 4	Fase 3: Modelado de datos en Swift y obtención de API Key externa.
Bloque 3	5 – 6	Fase 3: Desarrollo de vistas de catálogo y lógica de búsqueda.
Bloque 4	7 – 8	Fase 3: Gestión de favoritos y sincronización con base de datos MySQL.
Bloque 5	9 – 10	Fase 4: Pruebas de sistema, usabilidad y manejo de errores.
Bloque 6	11 – 12	Fase 5: Documentación final y entrega de informe de proyecto.

Gestión de Costos

- Presupuesto total: $0 USD.
- El proyecto opera bajo un modelo de costo cero, utilizando recursos institucionales y estaciones de trabajo personales.

Gestión de Cambios

La coordinación del proyecto se realizará mediante comunicación instantánea (WhatsApp) para garantizar una respuesta ágil ante cualquier modificación requerida durante el desarrollo.

Gestión de Calidad

Se validará el producto final bajo cuatro atributos clave:

1. Adecuación Funcional: Cumplimiento de registro, login, búsqueda y favoritos.
2. Eficiencia de Desempeño: Procesamiento de datos externos sin latencia excesiva.
3. Usabilidad: Interfaz nativa, fluida y adaptativa a diferentes orientaciones de pantalla.
4. Mantenibilidad: Código estructurado por módulos funcionales para facilitar correcciones.

FASE DE DISEÑO

La fase de diseño dentro del modelo MMS tiene como propósito transformar los requerimientos identificados en la planificación en soluciones concretas, tanto a nivel de arquitectura del sistema como de experiencia del usuario. En esta etapa se definen los modelos de datos, las interfaces y la estructura modular de la aplicación.

Objetivos de la Fase de Diseño

- Traducir los requerimientos funcionales y no funcionales en un diseño técnico concreto.
- Definir la arquitectura de la aplicación móvil y del backend.
- Diseñar el modelo de datos que soporte la lógica de negocio.
- Establecer los flujos de navegación y la interfaz de usuario.
- Garantizar que el diseño sea escalable, mantenible y alineado con las guías de estilo de iOS.
- Sentar las bases visuales y estructurales para la fase de desarrollo.

¿Quiénes Intervienen?

Participante	Rol en la Fase de Diseño	Responsabilidad Principal
Equipo Desarrollador	Diseñadores y arquitectos del sistema	Definición de arquitectura, modelos de datos, interfaces y flujos de navegación.
Docente (Supervisor)	Revisor y validador académico	Evaluación de decisiones de diseño y validación de entregables.
Usuarios Finales	Referente de usabilidad	Guiar decisiones de UX/UI a través de sus necesidades identificadas.
Administrador Backend	Diseñador de base de datos	Definición del modelo relacional y estructura de la API REST.

Actividades en la Fase de Diseño

- Análisis detallado de requerimientos funcionales y no funcionales.
- Reconocimiento y evaluación del problema desde la perspectiva del usuario.
- Observaciones múltiples sobre aplicaciones similares del mercado.
- Planteamiento de una solución ágil basada en los principios del Mobile Sprint.
- Modelado de la base de datos (MER y Modelo Relacional).
- Creación del diccionario de datos.
- Definición de la interfaz de usuario (wireframes y mockups).
- Diseño modular de la aplicación.

Análisis de Requerimientos

El análisis de requerimientos permite establecer con precisión qué debe hacer la aplicación y bajo qué condiciones debe operar. Se dividen en dos categorías:

Requerimientos Funcionales

ID	Requerimiento	Descripción
RF-01	Autenticación de usuarios	El sistema debe permitir el registro e inicio de sesión mediante correo y contraseña.
RF-02	Exploración de películas	El usuario puede visualizar un listado de películas con información básica (título, género, año, sinopsis).
RF-03	Búsqueda de películas	El sistema debe permitir buscar películas por título, género o año de estreno.
RF-04	Detalle de película	Al seleccionar una película se muestra información completa: título, sinopsis, género, director, reparto y calificación.
RF-05	Gestión de favoritos	El usuario autenticado puede agregar o eliminar películas de su lista de favoritos.
RF-06	Visualización de favoritos	El usuario puede consultar su listado personal de películas marcadas como favoritas.
RF-07	Cierre de sesión	El sistema debe permitir al usuario cerrar sesión de forma segura.

Requerimientos No Funcionales

ID	Requerimiento	Descripción
RNF-01	Rendimiento	La aplicación debe cargar el listado de películas en menos de 3 segundos bajo condiciones normales de red.
RNF-02	Usabilidad	La interfaz debe ser intuitiva, siguiendo las Human Interface Guidelines de Apple para iOS.
RNF-03	Seguridad	Las contraseñas deben almacenarse encriptadas y las sesiones deben manejarse con tokens JWT.
RNF-04	Compatibilidad	La aplicación debe ser compatible con iOS 15 o superior.
RNF-05	Escalabilidad	El backend debe estar diseñado para soportar futuras incorporaciones sin refactorización mayor.
RNF-06	Disponibilidad	El servicio backend debe tener una disponibilidad mínima del 95% durante el período de evaluación.

Reconocimiento y Evaluación del Problema

Dimensión del Usuario

El usuario objetivo es una persona que desea consultar información sobre películas (sinopsis, género, director, reparto) de forma rápida y sin necesidad de registrarse en plataformas de pago. Este perfil incluye estudiantes, cinéfilos casuales y personas que buscan decidir qué película ver sin comprometerse con una suscripción.

Dimensión Tecnológica

Las aplicaciones actuales como Netflix, Disney+ o Amazon Prime Video están optimizadas para la reproducción de contenido, no para la consulta informativa. Esto implica tiempos de carga elevados, interfaces sobrecargadas y una arquitectura diseñada para el streaming, no para la búsqueda y catalogación liviana.

Dimensión de la Solución

La aplicación propuesta aborda este problema al enfocarse exclusivamente en la consulta y gestión de información cinematográfica. Al eliminar funcionalidades innecesarias (reproducción, pagos, descargas), se obtiene una experiencia más fluida, ligera y centrada en el usuario.

Punto Crítico Identificado

El punto crítico del problema radica en la falta de una herramienta simple, gratuita y nativa para iOS que permita explorar y guardar información de películas sin requerir suscripción ni generar una carga cognitiva elevada en el usuario.

Observaciones Múltiples

Como parte del proceso de diseño centrado en el usuario, se realizó un análisis comparativo de aplicaciones existentes con funcionalidades similares:

Aplicación	Plataforma	Fortalezas Observadas	Debilidades Observadas
IMDb	iOS / Android	Información completa, búsqueda avanzada, base de datos extensa.	Interfaz sobrecargada, exceso de publicidad, funciones de streaming innecesarias.
Letterboxd	iOS / Android	Diseño minimalista, gestión de listas, comunidad activa.	Requiere registro, algunas funciones son de pago.
JustWatch	iOS / Android	Filtros avanzados, multiplataforma.	Enfocado en disponibilidad de streaming, no en información pura.
Filmaffinity	Web / iOS	Calificaciones detalladas, búsqueda por género.	Diseño desactualizado, experiencia móvil deficiente.

Conclusiones de las Observaciones

- Existe una oportunidad de mercado para una aplicación iOS nativa, ligera y gratuita orientada exclusivamente a la consulta informativa.
- El diseño debe seguir las Human Interface Guidelines de Apple para garantizar coherencia y familiaridad.
- La búsqueda y la gestión de favoritos son las funcionalidades más valoradas en este tipo de aplicaciones.

Planteamiento de Solución Ágil

Descripción General de la Solución

Se desarrollará una aplicación móvil nativa para iOS denominada CineInfo, la cual permitirá a los usuarios explorar, buscar y gestionar información de películas de forma gratuita, rápida e intuitiva, sin necesidad de suscripción ni funciones de reproducción.

Arquitectura de la Solución

La solución adoptará una arquitectura Cliente–Servidor con los siguientes componentes:

- Cliente (iOS): Aplicación nativa en Swift con SwiftUI, siguiendo el patrón MVC. Se comunica con el backend mediante peticiones HTTP a la API REST.
- Servidor (Backend): API REST desarrollada con Python y Flask, que expone endpoints para autenticación, consulta de películas y gestión de favoritos.
- Base de Datos: MySQL, gestionada en el servidor backend, almacena la información de películas, usuarios y favoritos.

Plan de Sprints

Sprint	Duración	Objetivo Principal	Entregables
Sprint 0	1 semana	Planificación y diseño del sistema	Documento de fases, modelos de datos, wireframes
Sprint 1	2 semanas	Configuración del entorno y autenticación	Proyecto Xcode, API Flask, módulo de login/registro
Sprint 2	2 semanas	Exploración y búsqueda de películas	Pantalla de listado, búsqueda funcional, detalle de película
Sprint 3	2 semanas	Gestión de favoritos e integración final	Módulo de favoritos, integración completa, pruebas
Sprint 4	1 semana	Revisión, pruebas y documentación final	APK de prueba, documentación técnica completa

Criterios de Éxito de la Solución Ágil

- Cada sprint finaliza con un entregable funcional y evaluable.
- Se realizan revisiones semanales con el docente para validar el avance.
- Los cambios se gestionan a través del control de cambios definido en la planificación.
- La aplicación final cumple con todos los requerimientos funcionales y no funcionales establecidos.

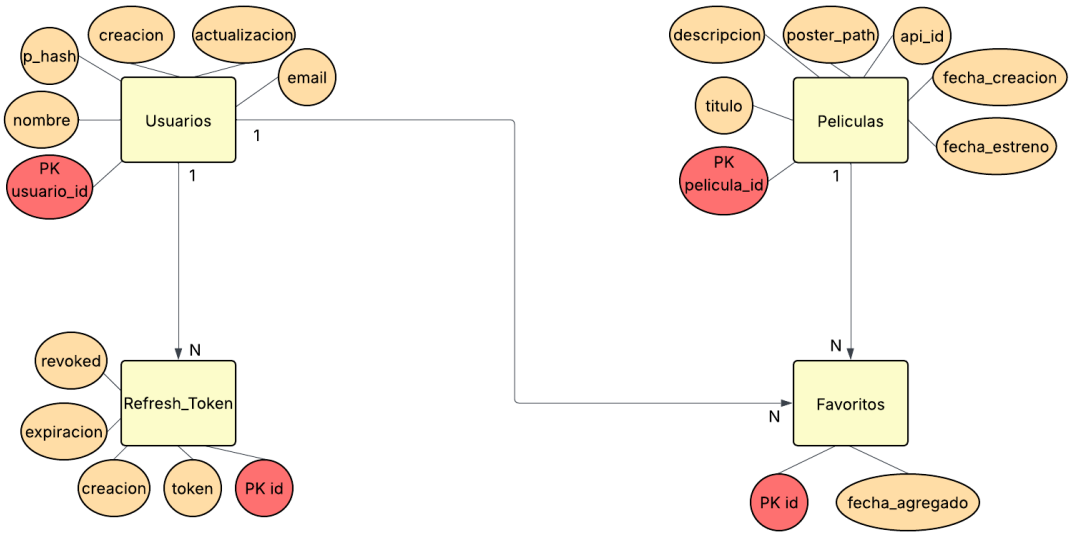
Modelado de la Base de Datos

Para soportar la arquitectura Cliente-Servidor y garantizar la persistencia de la información, se ha diseñado una base de datos relacional con MySQL, orientada a gestionar la autenticación segura mediante tokens JWT, almacenar una copia local (caché) de los datos de películas consultadas a la API de TMDB, y administrar los favoritos de los usuarios.

Modelo Entidad-Relación (MER)

El MER conceptualiza las entidades principales del sistema y cómo se relacionan entre sí:

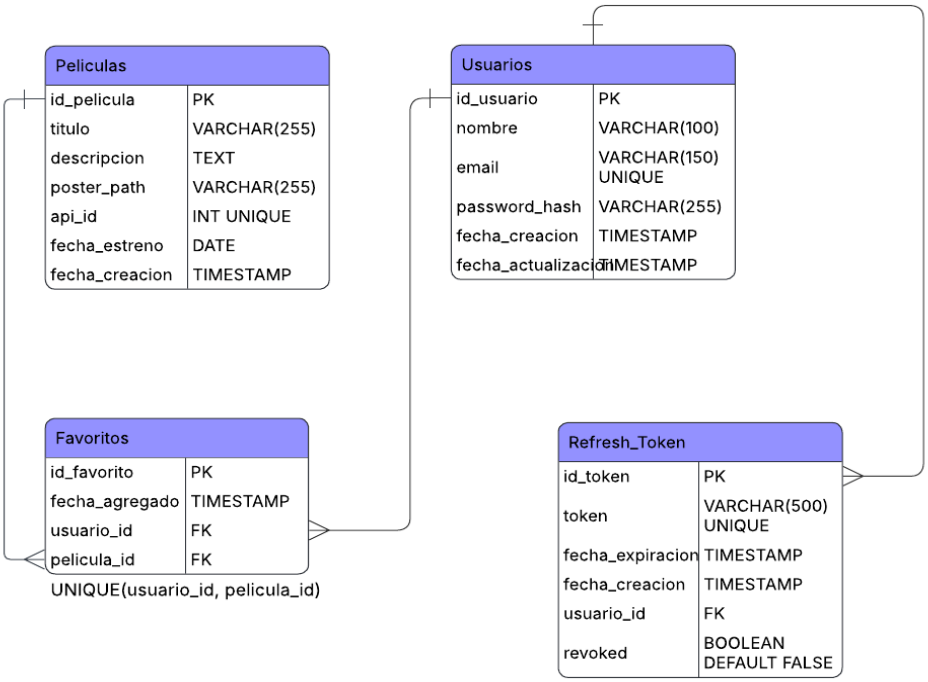
- Usuarios: Representa a las personas registradas en el sistema.
- Refresh_Token: Entidad débil dependiente de "Usuarios" para la gestión de sesiones. Relación 1:N.
- Peliculas: Catálogo de películas obtenido desde la API externa (TMDB).
- Favoritos: Entidad asociativa que resuelve la relación M:N entre "Usuarios" y "Peliculas".



Modelo Relacional (MR)

El Modelo Relacional traduce el MER en la estructura de tablas, claves primarias (PK), claves foráneas (FK) y restricciones de integridad para su implementación en MySQL:

- Seguridad: La tabla Usuarios almacena password_hash en lugar de texto plano.
- Integridad de Datos: Restricción UNIQUE en Favoritos para (usuario_id, pelicula_id) para prevenir duplicados.
- Sincronización API: El campo api_id en Peliculas tiene restricción UNIQUE para correspondencia exacta con los IDs de TMDB.



Diccionario de Datos

El diccionario de datos define de manera exhaustiva cada tabla, sus campos, tipos de datos y restricciones del Modelo Relacional.

Tabla 1: Usuarios

Almacena las credenciales y el perfil de los usuarios registrados.

Campo	Tipo de Dato	Restricción / Clave	Descripción
id_usuario	INT	PK, Auto Increment	Identificador único del usuario.
nombre	VARCHAR(100)	NOT NULL	Nombre completo o alias del usuario.
email	VARCHAR(150)	UNIQUE, NOT NULL	Correo electrónico para registro y login.
password_hash	VARCHAR(255)	NOT NULL	Contraseña del usuario encriptada (hash).
fecha_creacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha y hora de registro del usuario.
fecha_actualizacion	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Fecha y hora de la última modificación del perfil.

Tabla 2: Refresh_Token

Gestiona los tokens de sesión de larga duración para la autenticación basada en JWT.

Campo	Tipo de Dato	Restricción / Clave	Descripción
id_token	INT	PK, Auto Increment	Identificador único del registro del token.
token	VARCHAR(500)	UNIQUE, NOT NULL	Cadena de texto que representa el refresh token.
fecha_expiracion	TIMESTAMP	NOT NULL	Fecha y hora en la que el token pierde validez.
fecha_creacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha y hora de emisión del token.
usuario_id	INT	FK	Llave foránea vinculada a la tabla Usuarios.
revoked	BOOLEAN	DEFAULT FALSE	Bandera para invalidar el token de forma manual o por seguridad.

Tabla 3: Peliculas

Funciona como caché local de los datos extraídos de la API de TMDB para minimizar peticiones externas y acelerar la carga.

Campo	Tipo de Dato	estricción / Cla	Descripción
id_pelicula	INT	PK, Auto Increment	Identificador único interno de la base de datos.
titulo	VARCHAR(255)	NOT NULL	Título oficial de la película.
descripcion	TEXT	—	Sinopsis o descripción de la trama.
poster_path	VARCHAR(255)	—	Ruta o sufijo de la URL del póster proporcionado por la API.
api_id	INT	UNIQUE, NOT NULL	Identificador oficial de la película en la API de TMDB.
fecha_estreno	DATE	—	Fecha en la que la película fue lanzada.
fecha_creacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha en la que la película se guardó en la base local.

Tabla 4: Favoritos

Tabla intermedia que establece la relación entre los usuarios y sus películas guardadas.

Campo	Tipo de Dato	Restricción / Clave	Descripción
id_favorito	INT	PK, Auto Increment	Identificador único del registro de favorito.
fecha_agregado	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha y hora en la que el usuario marcó la película.
usuario_id	INT	FK	Llave foránea referenciando a Usuarios.
pelicula_id	INT	FK	Llave foránea referenciando a Peliculas.

Nota técnica: *La tabla Favoritos posee una restricción compuesta de tipo UNIQUE(usuario_id, pelicula_id) para garantizar que un mismo usuario no pueda insertar registros duplicados para la misma película.*

Definición de Interfaz

La interfaz de usuario se diseñará siguiendo las Human Interface Guidelines (HIG) de Apple, garantizando una experiencia nativa, fluida y consistente para el usuario de iOS. Los principios rectores son:

- Claridad: Texto legible, iconos precisos y elementos bien organizados.
- Deferencia: La interfaz apoya el contenido sin competir con él visualmente.
- Profundidad: Jerarquía visual clara con capas y transiciones significativas.

Diseño Modular

La aplicación se estructura en módulos independientes que facilitan el desarrollo, mantenimiento y escalabilidad del sistema.

¿Qué se considera en los módulos?

Módulo	Descripción
Autenticación	Registro, login y cierre de sesión con gestión de tokens JWT.
Catálogo	Listado de películas con paginación, búsqueda y filtros.
Detalle	Visualización completa de información de una película seleccionada.
Favoritos	Gestión de la lista personal del usuario con persistencia en MySQL.
Red	Capa de comunicación con la API REST y la API externa de TMDB.
Persistencia	Manejo de datos locales y sincronización con el backend.

CONCLUSIONES Y PRÓXIMOS PASOS

El desarrollo de las fases de Planificación y Diseño de la aplicación CineInfo mediante la metodología Mobile Sprint Model (MMS) ha permitido establecer una base sólida y estructurada para el proyecto. Se logró identificar con claridad el problema, definir los requerimientos, analizar la factibilidad y diseñar la arquitectura del sistema.

Los modelos de datos diseñados garantizan la integridad, seguridad y escalabilidad del backend, mientras que el diseño modular de la aplicación facilita el desarrollo iterativo por sprints.

Próximos Pasos

5. Iniciar el Sprint 1 con la configuración del entorno de desarrollo y la implementación del módulo de autenticación.
6. Configurar el repositorio en GitHub y establecer el flujo de trabajo colaborativo.
7. Implementar la API REST con Flask y conectar con la base de datos MySQL.
8. Desarrollar las primeras vistas en SwiftUI siguiendo el diseño definido.
9. Realizar pruebas de integración al finalizar cada sprint para garantizar la calidad del producto.

FUENTES CONSULTADAS

- Apple Inc. (2024). Human Interface Guidelines. https://developer.apple.com/design/human-interface-guidelines/
- The Movie Database (TMDB). (2024). API Documentation. https://developer.themoviedb.org/docs
- Palacio, J. (2022). Metodología Mobile Sprint para el desarrollo de aplicaciones móviles. Editorial Scrum Manager.
- Sommerville, I. (2016). Ingeniería del Software (10.ª ed.). Pearson Educación.
- Pressman, R. & Maxim, B. (2014). Ingeniería del Software: Un enfoque práctico (8.ª ed.). McGraw-Hill.